

The Classical Thread Model

Definition

The **classical thread model** separates two ideas that are combined in a traditional process: **resource grouping** and **execution**. A process groups resources, while a thread is the unit of execution scheduled on the CPU.

1. Two Concepts Inside a Process

Resource Grouping

One role of a process is to collect related resources together. A process may contain:

- an address space,
- program text and data,
- open files,
- child processes,
- pending alarms,
- signal handlers,
- accounting information.

Execution

The second role is execution. A **thread** is the entity that actually runs on the CPU. It has:

- a program counter,
- registers,
- and a stack.

Core Separation

Processes are used to group resources together. Threads are the execution units scheduled by the CPU.

2. What Threads Add

Main Idea

Threads allow multiple executions to take place within the same process environment. These threads run largely independently, but they share the same address space and process resources.

Lightweight Processes

Because threads behave in some ways like processes, they are often called **lightweight processes**. The term **multithreading** means allowing multiple threads inside one process.

Hardware Support

Some CPUs provide direct hardware support for multithreading, allowing thread switches to happen extremely quickly.

3. Processes vs Threads

Feature	Traditional Processes	Threads in One Process
Address space	Each process has its own	All threads share one address space
Resources	Separated per process	Shared inside the process
Execution flows	One per process in the simple model	Multiple inside the same process
Cooperation	Harder because memory is separate	Easier because memory is shared

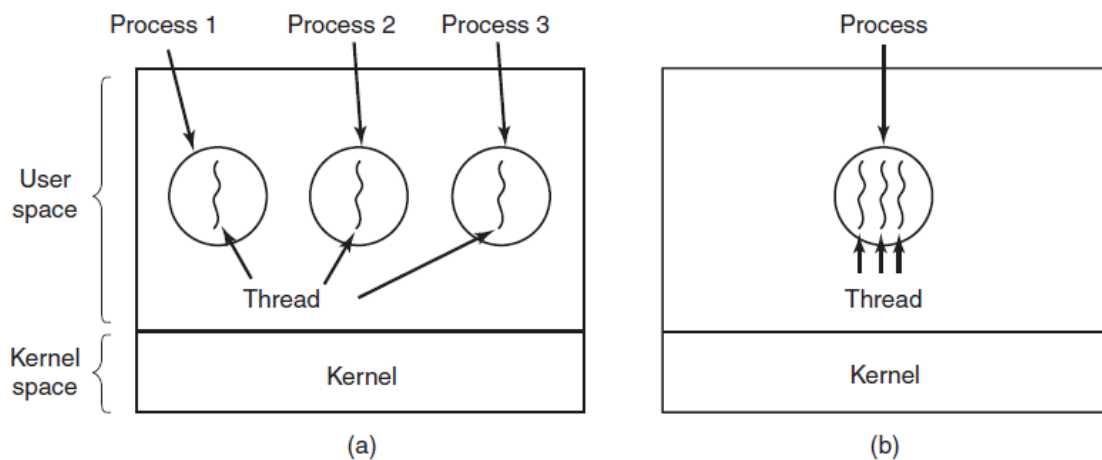


Figure 1: Three processes each with one thread, and one process with three threads.

Important Difference

Threads are not as independent as processes. Since all threads share the same address space, one thread can read or modify data used by another thread.

4. Shared and Private Items

Shared vs Private

Some items belong to the whole process, while others belong to each individual thread.

Per-process items	Per-thread items
Address space, global variables, open files, child processes, pending alarms, signals and signal handlers, accounting information	Program counter, registers, stack, state

Why This Matters

If one thread opens a file, the other threads in the same process can use that file too. This happens because the process, not the thread, is the unit of resource management.

Why No Protection Exists

There is normally no memory protection between threads of the same process because they are expected to cooperate, not compete like unrelated processes.

5. How Threads Run on a Single CPU

Quasi-Parallelism

On a single-CPU system, multiple threads do not literally run at the same instant. Instead, the CPU switches rapidly among them, creating the illusion of parallel execution.

Example

If one process has three compute-bound threads on a single CPU, they appear to run in parallel, but each one effectively gets about one-third of the CPU time.

6. Thread States

States

Like processes, threads can be in several states:

- **Running**
- **Ready**
- **Blocked**
- **Terminated**

Meaning

- A **running** thread currently has the CPU.
- A **ready** thread can run when scheduled.
- A **blocked** thread is waiting for some event.
- A **terminated** thread has finished and is no longer schedulable.

Same Transition Idea

The transitions among thread states are conceptually the same as the transitions among process states.

7. Why Each Thread Needs Its Own Stack

Private Execution History

Each thread has its own stack because each thread may be executing a different sequence of procedure calls.

Stack Frames

Each stack contains one frame for every procedure that has been called but not yet returned from. A frame stores:

- local variables,
- return addresses,
- and related execution data.

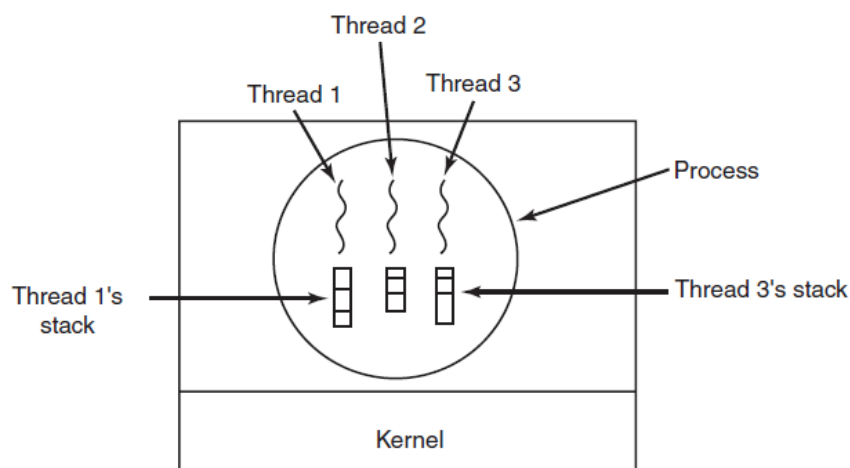


Figure 2: Each thread has its own stack.

Example

If thread A is inside procedures $X \rightarrow Y \rightarrow Z$, while thread B is inside $P \rightarrow Q$, then the two threads have different execution histories and therefore need separate stacks.

8. Basic Thread Operations

Starting Point

A multithreaded process usually begins with one thread. That thread can create additional threads.

Common Thread Calls

Typical thread library operations include:

- `thread_create`
- `thread_exit`
- `thread_join`
- `thread_yield`

Typical Usage

```
thread_create(worker_function);
thread_create(logger_function);

thread_yield();

thread_join(worker_id);
thread_exit();
```

What They Mean

- `thread_create`: starts a new thread in the same address space.
- `thread_exit`: ends the current thread.
- `thread_join`: waits for a specific thread to finish.
- `thread_yield`: voluntarily gives up the CPU.

Polite Scheduling

Voluntary yielding is especially important in thread systems where threads are expected to cooperate and give other threads a chance to run.

9. Complications Introduced by Threads

9.1 Problem 1: Fork

Fork Ambiguity

If a multithreaded process calls `fork`, an important question arises: should the child process receive all of the parent's threads or only the calling thread?

Why This Is Difficult

If only one thread is copied, the program may break because other threads may be essential. If all threads are copied, blocked operations such as keyboard reads or network reads become hard to interpret correctly in both parent and child.

9.2 Problem 2: Shared Data Structures

Race Risks

Because threads share data, one thread can interfere with another in dangerous ways.

Examples

- One thread may close a file while another is still reading from it.
- Two threads may both notice low memory and both start allocating memory, causing duplication or corruption.

Design Requirement

Multithreaded programs can work correctly, but they require careful design because shared memory and shared resources create synchronization problems.

10. Main Conclusion

Takeaway

The classical thread model separates **resource ownership** from **execution**. A process owns resources, while threads execute inside that process. This gives efficient cooperation through shared memory, but it also introduces correctness and synchronization problems.

11. Quick Review

Exam-Focused Points

- A traditional process combines resource grouping and execution.
- The classical thread model separates these two ideas.
- A process owns resources; a thread is the unit scheduled on the CPU.
- Threads in the same process share address space and process resources.
- Each thread has its own program counter, registers, stack, and state.
- Threads can be running, ready, blocked, or terminated.
- Each thread needs its own stack because each has its own execution history.
- Common thread operations include `thread_create`, `thread_exit`, `thread_join`, and `thread_yield`.
- Multithreading introduces complications such as `fork` ambiguity and races on shared data.